
BTF Viewer — Application Notes

Practical, **top-down** workflows for analysing RTOS scheduler behaviour with BTFViewer. Start at the system level, follow the Analysis findings, and only then open the deep metric tables.

Shell examples assume the current directory is BTFViewer/ (Desktop: python builds/btf_viewer.py ...). The Web viewer is builds/btf_viewer.html.

The worked example throughout uses the committed sample:

```
python builds/btf_viewer.py ../tracedata/example-8cores.btf.gz
# Headless report (numbers cited below come from this):
python builds/btf_viewer.py report ../tracedata/example-8cores.btf.gz \
    --output /tmp/ex8-report.html --format html
```

Table of Contents

Section	Purpose
1. Load a trace	Open .btf / .btf.gz and orient
2. Top-down analysis ladder	The order to inspect any new trace
3. Worked example — example-8cores	Full walkthrough with measured stats and recommendations
4. Deep-dive playbooks	Procedures once a finding points at a metric
5. Scope, compare, and custom signals	Cursor windows, Trace Compare, tags/intervals
6. Export results	Analysis Findings, CSV/HTML, Perfetto, CLI
Quick-reference	Symptom → metric map

1. Load a Trace

```
python builds/btf_viewer.py ../tracedata/example-8cores.btf.gz
# Smaller smoke sample:
python builds/btf_viewer.py ../tracedata/example-2cores.btf.gz
```

Or **File** → **Open** (Ctrl+O) / drag-and-drop. Plain .btf and compressed .btf.gz / .bz2 / .zip all work. Web: open builds/btf_viewer.html (or **Demo** for the embedded example-2cores.btf.gz).

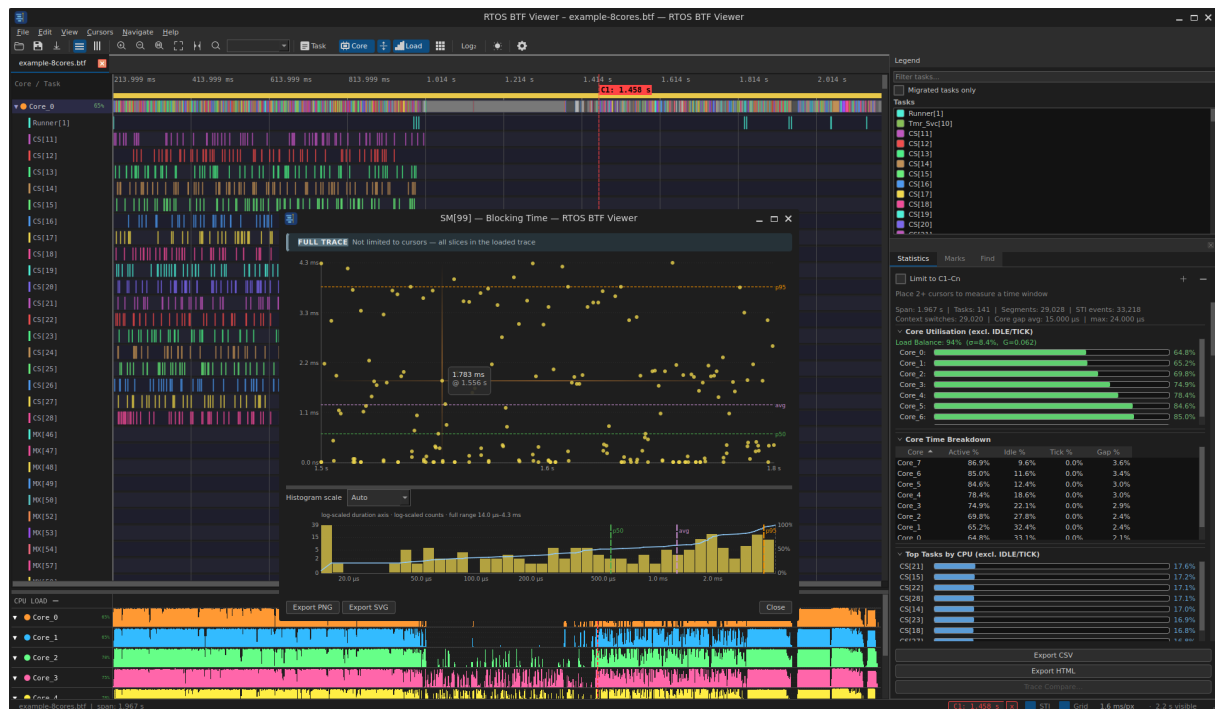


Figure 1: BTF Viewer — Core View with CPU Load graph and Statistics

Core View + CPU Load + Statistics. Prefer Core View on first open of an SMP trace.

Step	Action	What to look for
1	Status bar + Ctrl+O	Span, task/segment counts; Fit to Window
2	Toolbar Core + Load	Core View; CPU Load strip (one row per core)

Step	Action	What to look for
3	Click a task label (lock-highlight)	Task hops across core rows; CPU Load shows that task's usage per core
4	Statistics + Analysis	Summary + heuristic triage
5	Trace Health (TICK)	Timer health before trusting derived metrics

Fit + Core View + highlight + per-core CPU Load (headless)

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
  -o ../images/stats/tasks-cpu-load-cs22.svg \
  --view timeline --view-mode core --task "CS[22]" --cpu-load --height 900
```

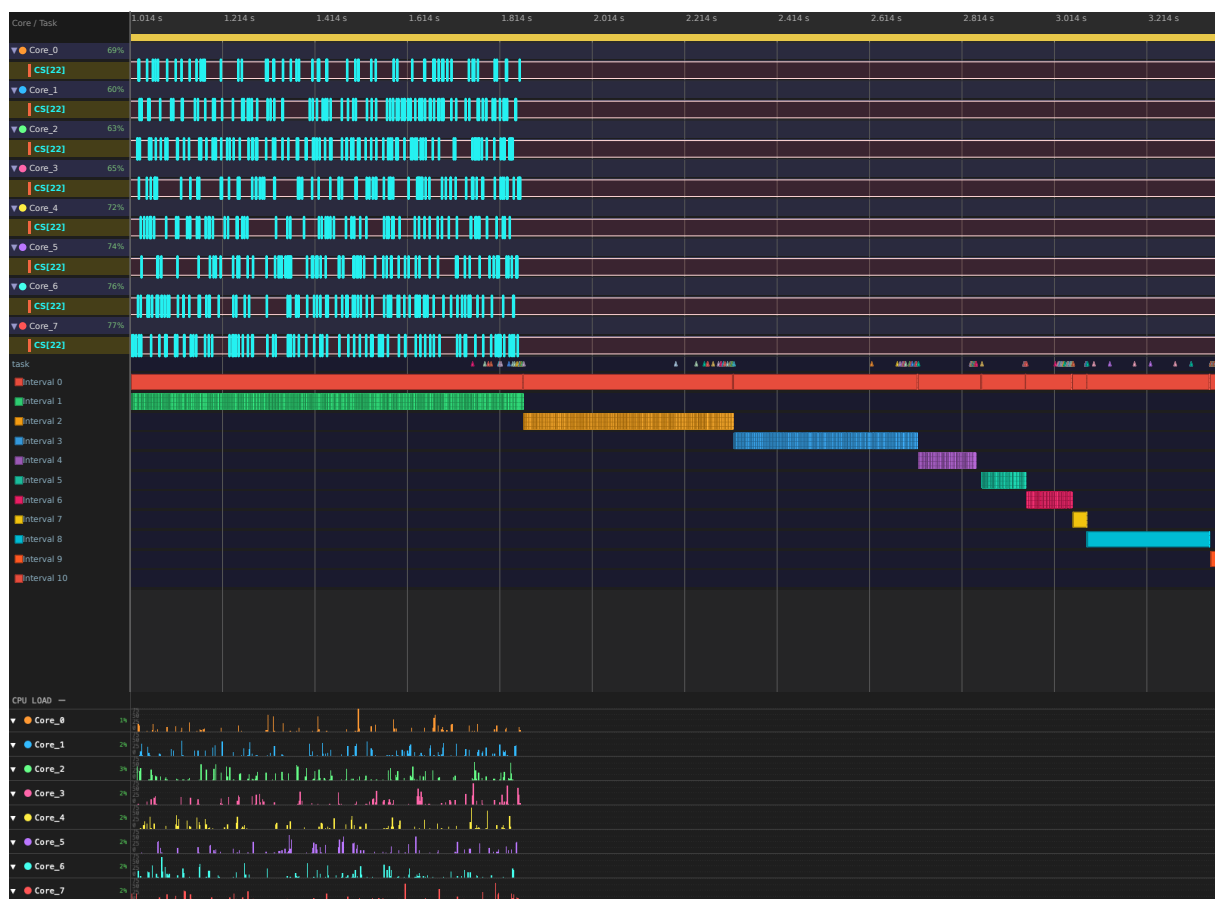


Figure 2: CS[22] in Core View with per-core CPU Load at Fit to Window

*CS[22] locked in Core View. Timeline shows the task on each core it ran; **CPU Load** has eight sparklines (that task's load on Core_0...Core_7).*

2. Top-Down Analysis Ladder

Work **down** this hierarchy. Stop drilling when the evidence explains the symptom; do not open every Statistics section by default.

□ System health	Trace Health (TICK), span, instrumentation flags
□ Core balance	Core Utilisation + Load Balance Score
□ Task CPU / WCET	Top Tasks → Execution Time Per Slice (Max)
□ Latency	Blocking Time → Preemption Chain
□ Concurrency	Core Migrations → Mutex/Semaphore → Priority Inheritance
□ Compliance	Deadlines / Affinity / Task Lifecycle / Tags / Intervals

How to drive it in the UI

1. Toolbar **Analysis** — read every finding (info and warning).
2. For each finding, open the named Statistics section.
3. Click **Max** / scatter points / heatmap cells to jump the timeline.
4. Place cursors (C) around the interesting phase and enable **Limit to cursor range** so metrics are not polluted by unrelated tests in the same file.

Interpretation rule for demo traces: example-8cores.btf.gz concatenates intentional stress tests (context-switch thrash, mutex/queue storms, priority inversion, suspend/resume, affinity). A “warning” finding may be **expected for that test phase**. Scope to the phase before treating it as a product defect.

3. Worked Example — example-8cores

3.0 Trace snapshot

Field	Value
File	../tracedata/example-8cores.btf.gz
Span	2.358 s (#timeScale us)
Cores	8
Tasks / segments / STI	154 / 31 141 / 33 495
Context switches / migrations	31 133 / 18 992 (138 migrated tasks)
Instrumentation	priority + sync + intervals
TICK	WARNING / TICKLESS — 2496 ticks, CV 35.9 % , missed $\approx$ 8

```
python builds/btf_viewer.py info ../tracedata/example-8cores.btf.gz
```

Open **Statistics**, then **Analysis**. The HTML report's Analysis card summarises the same triage used below.

3.1 System health (TICK)

Statistics → Trace Health (TICK)

Metric	Measured
Mode	TICKLESS (CV 35.9 % $\gg$ 5 % threshold)
Avg period	944 μ s (nominal 1.000 ms)
Max gap	2.481 ms
Missed ticks (est.)	8 large gaps

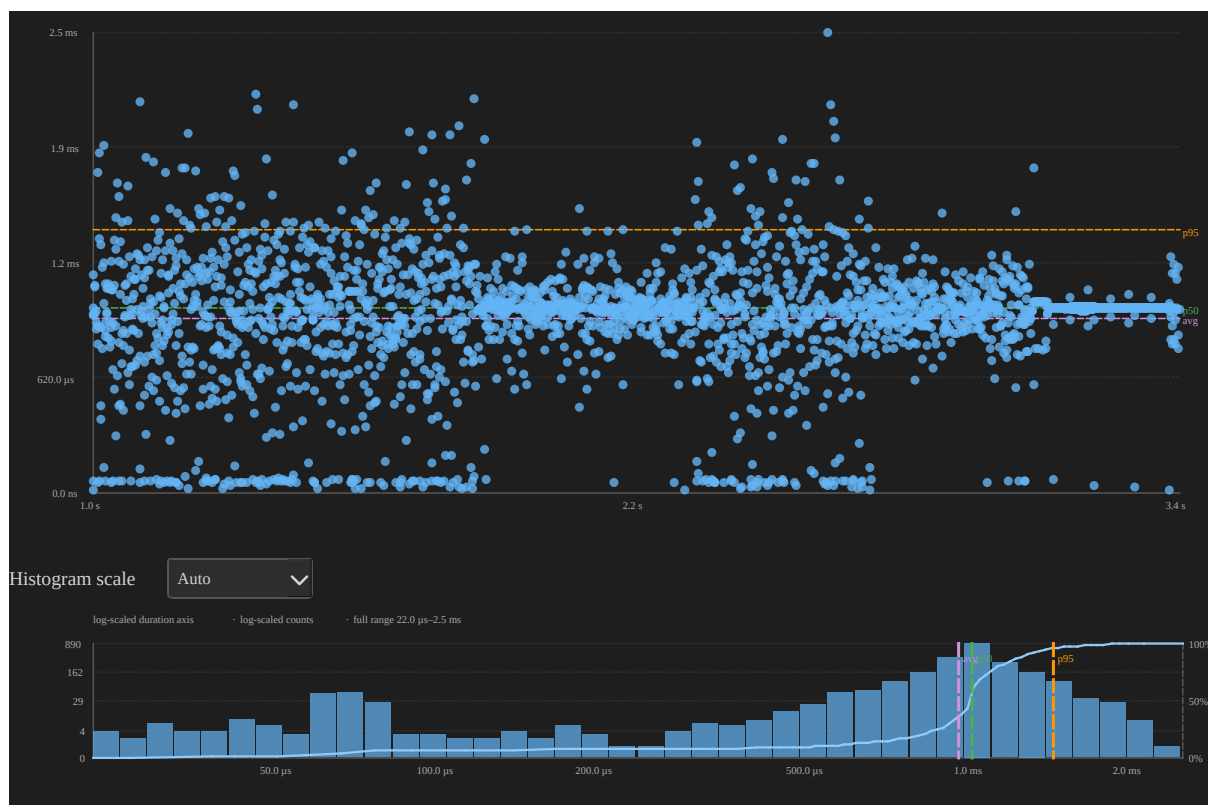


Figure 3: Tick interval distribution — example-8cores

2496 TICK events; multi-tick gaps dominate while cores idle between suite phases.

Reading: Tickless idle is active. Absolute wall-clock gaps between ticks are **not** proof of a lost interrupt during busy work — many gaps sit in idle stretches between tests.

Recommendations

- For timing conclusions, **Limit to cursor range** on a busy phase (e.g. CS stress) and re-check CV / large gaps.
- If the product must run tickful, disable tickless in the board config and re-capture; expect $CV \ll 5\%$.
- Keep TICK STI enabled so Trace Health remains trustworthy.

3.2 Core balance

Statistics → Core Utilisation

Core	CPU % (excl. IDLE/TICK)
Core_0 ... Core_7	68.7, 59.9, 62.7, 65.1, 71.6, 74.0, 76.1, 77.3
Load Balance Score	95 % (G = 0.049)
σ	6.0 % (gauge green / OK; amber only if $\sigma > 30 %$; red if Score < 70 %)

Analysis finding: *cores look reasonably balanced* (Score $\geq 85 %$ and $\sigma \leq 30 %$).

Reading: SMP placement is healthy at the system level. Imbalance is **not** the primary problem on this trace.

Recommendations

- No affinity rebalance needed for overall util.
- Still inspect migrations (next) — balance can coexist with expensive bouncing.

3.3 Task CPU and WCET

Top Tasks by CPU (excl. IDLE/TICK) are the equal-priority context-switch workers CS[11]...CS[28]:

Task	CPU %	Runs	Max slice	p95
CS[28]	15.9 %	728	3.623 ms	1.631 ms
CS[11]	15.3 %	730	2.486 ms	1.628 ms
CS[24]	15.3 %	727	3.283 ms	1.644 ms
...	~14-15 % each	~700+	~2.5-3.5 ms	~1.5 ms

Also notable: Med[267] 6.0 % CPU (priority-inversion medium task), SM[*] / MX[*] mutex/semaphore workers ~5-6 %.

Procedure: Execution Time Per Slice → sort by **Max** → click **Max** to zoom and annotate the WCET slice.

Reading: CPU is dominated by the intentional CS stress cohort. Worst slices (~3-4 ms) are long relative to a 1 ms tick — they contribute to tick stretching when those tasks run.

Recommendations

- **Production:** do not ship unbounded equal-priority worker storms; cap concurrency, or pin workers with `vTaskCoreAffinitySet` so the scheduler does less cross-core juggling.
- Set **Settings** → **Analysis thresholds** deadlines (e.g. CS[28]=20000000 for 2 ms) and re-run Statistics → Deadlines to turn WCET into pass/fail.
- Click Max on CS[28] and check whether the long slice coincides with a lock hold or a migration (Preemption Chain / Mutex).

3.4 Latency (blocking / response time)

Statistics → **Blocking Time** (off-CPU gap = Tracealyzer-style response-time gap)

Task	Gaps	Max block	Notes
Med[267]	1160	35.305 ms	Busy medium task around inversion test
Low[266]	725	39.718 ms	Low holder; long waits around boost windows
High[268]	6	52.865 ms	High waiter blocked during inversion demo

Task	Gaps	Max block	Notes
CS[*]	~700+	~5–6 ms	Contended equal-priority cohort
Runner[1]	335	737 ms	Orchestrator sleeping between tests (ignore)

Analysis finding: warning on blocking candidates (Med[267], CS[19], ...).

Reading: The extreme High/Low/Med gaps are **features of test 8** (priority inversion), not random glitches. CS blocking ~5 ms is contention + migration among peers.

Recommendations

- Scope cursors to the inversion window (Low/Med/High alive ~3.085–3.310 s) when studying PI; ignore Runner’s multi-hundred-ms sleeps.
- For product tasks with High-like latency, require mutex priority inheritance (see §3.5) and measure Max block under load.
- Cross-check **Preemption Chain**: top pairs are CS[*] preempting CS[*] (peer interference), e.g. CS[25]←CS[19] 44× / 14 ms total.

3.5 Concurrency — migrations, locks, priority inheritance

Core migrations (thrashing) **Analysis finding:** *Excessive bouncing / core thrashing* on CS tasks.

Task	Migrations	Rate	Avg dwell	Ping-pong	Cores
CS[18]	586	1692 /s · 1.41/tick	476 μs	24	8
CS[21]	580	1648 /s	494 μs	19	8
CS[12]	579	1615 /s	494 μs	23	8

Trace-wide: **18 992** migrations. Hottest pairs (Core-Pair Summary): Core_5→Core_7 (599), Core_4→Core_6 (560), ... — mostly **0 % lock-bounce** on those pairs.

Reading: Expected outcome of test 1 (equal-priority CS storm on all cores). Cache-cold penalties and run-queue churn would dominate a real product with this pattern.

See it on the timeline — Core View + task filter + lock-highlight (same window as the doc image):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
-o ../images/stats/tasks-migrate-cs22.svg \
--view timeline --view-mode core --task "CS[22]" \
--lo 1805000 --hi 1865000
```

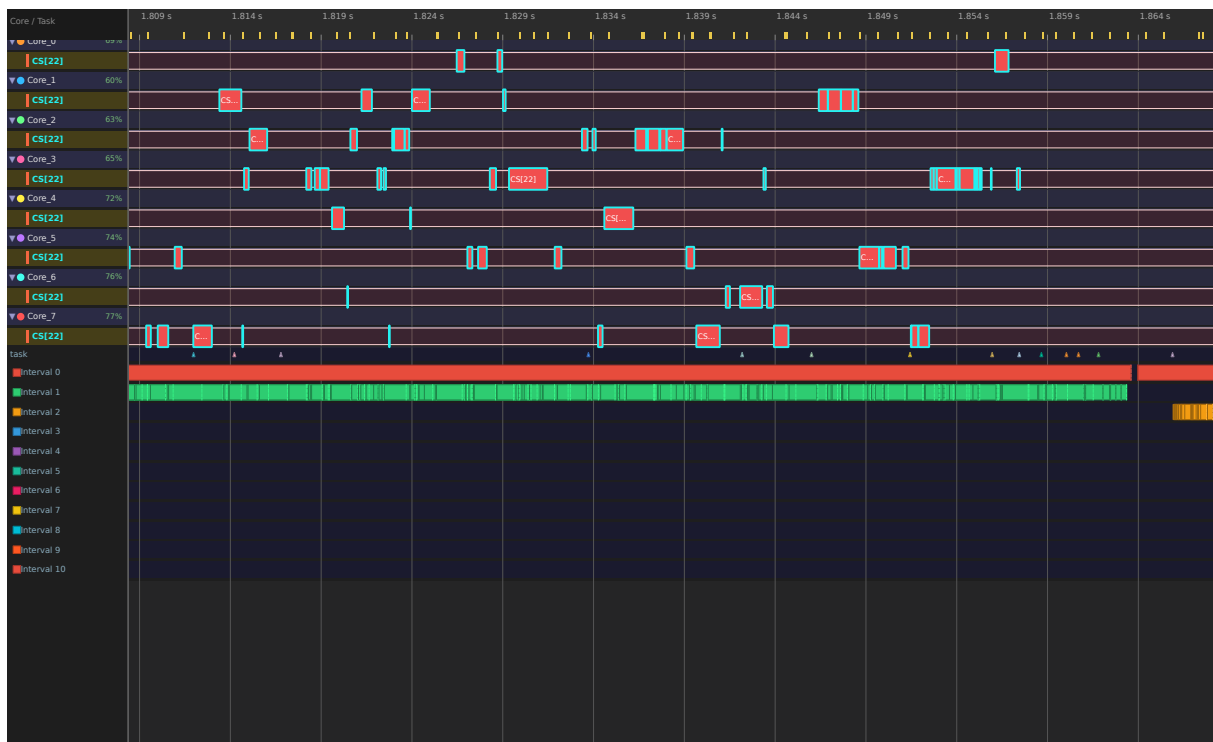


Figure 4: CS[22] highlighted across Core_0...Core_7

CS[22] hopping all eight cores in ~60 ms. Then open **Core Migrations** for that task, **Core-Pair Migration Summary** for pairs involving its **Primary** core, and toolbar **Heatmap / Chord** to inspect traffic on a specific core.

Recommendations

-
- Pin latency-critical tasks to a core mask; leave only best-effort work fully migratable.
 - Prefer fewer runnable equals over more workers “for throughput.”
 - Use toolbar **Heatmap** (32 time bins) to confirm migrations concentrate in the CS phase, then scope Statistics to that window.
 - For a single core’s fan-in/fan-out: Chord hover on that core’s arc, or Heatmap rows where the core is From/To.

Mutex / semaphore / queue

Object	Holds	Issues	Core bounces	Status
mutex 0x80021920	864	6	6	Warning
queue 0x80021990	864	0	858	OK status, but extreme bounce
several sems	few	1 each	2-5	Warning (unmatched / bounce)

Analysis finding: 19 sync objects with Core bounce > 0; 6 migration-while-held style issues.

Recommendations

- Tasks that share a hot mutex/queue should share an **affinity mask** so holds do not ride migrations (queue with 858 bounces is the smoking gun).
- Investigate Warning rows (unmatched take/delete) on the timeline; demo tear-down can leave unpaired STI — confirm against firmware if the same pattern appears outside the suite.

Priority inheritance

Task	Base→Peak	Boosts	Boosted time	Pattern
Low[266]	2→4	3	103.318 ms	Mutex inherit
PS[228]	2→4	1	119 μ s	L/M/H pattern

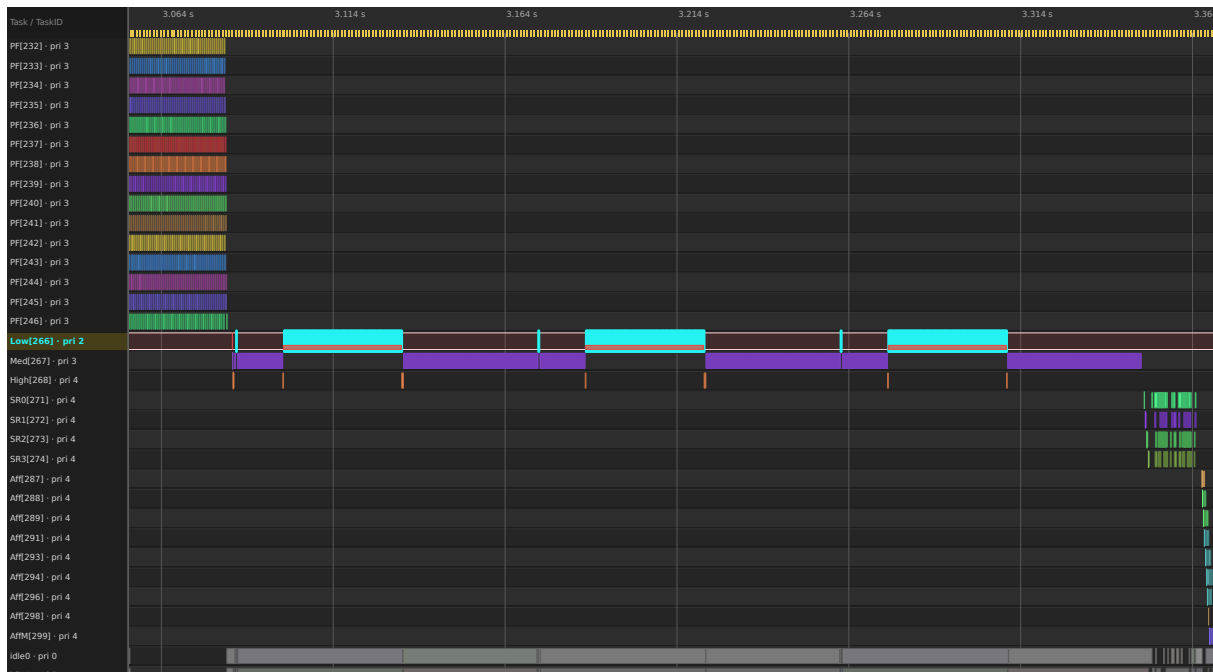


Figure 5: Priority inversion — red inherit stripes on Low

Three red bottom stripes on Low[266] during test 8 (`priority_inherit` → `priority_disinherit`).

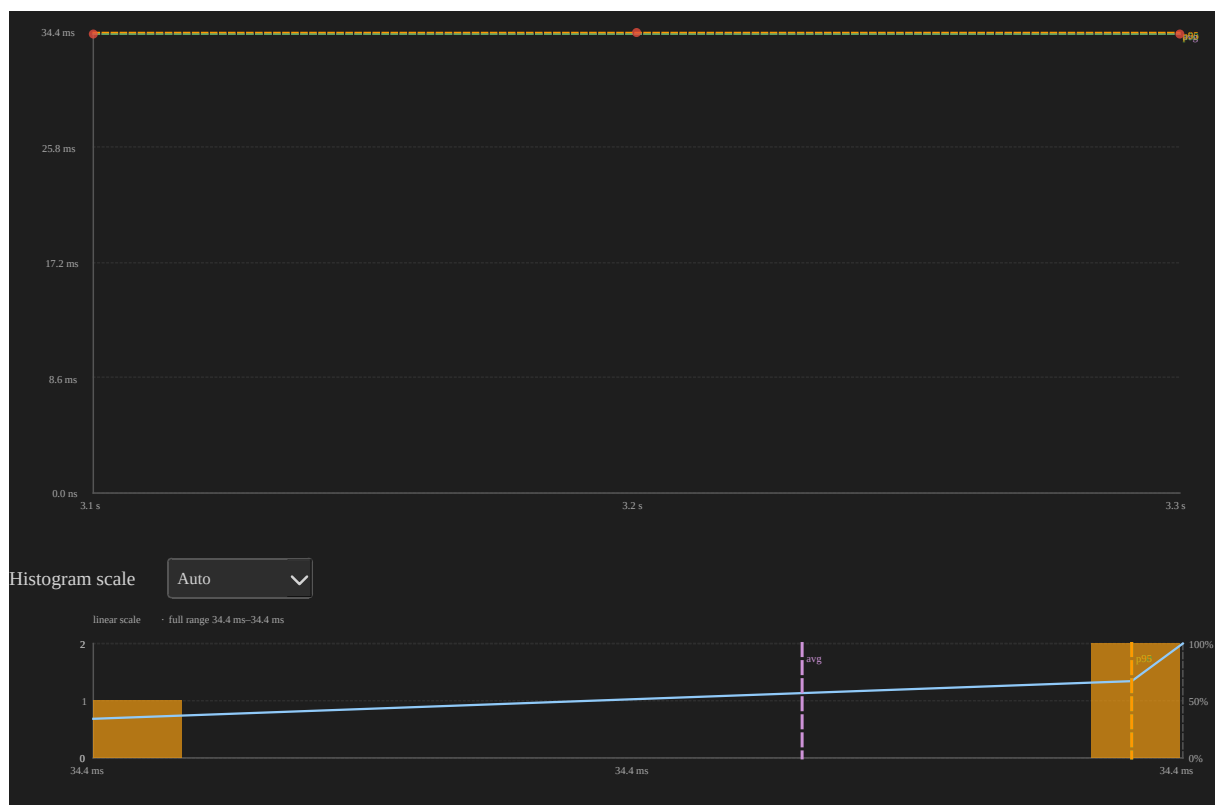


Figure 6: Priority boost distribution

Reading: Kernel inheritance on Low[266] is working (correct mitigation). PS[228] still shows L/M/H geometry — open that episode (click the chart point) and confirm whether a medium task ran while a higher waiter blocked.

Recommendations

- Keep priority inheritance enabled for mutexes that can block higher priorities.
- Treat any **L/M/H** row as a design review item (shorten critical sections, raise holder base priority, or split locks).
- Snapshot window used in docs: `--lo 3042000 --hi 3359000` (μ s) around the inherit stripes.

3.6 Compliance — affinity, suspend/resume, tags

Core affinity (test 10) Statistics → Core Affinity — Aff[287]...Aff[298] masks 0x1...0x80 observed only on the matching core; AffM[299] mask changes 0x1 → 0x80 and is seen on Core_0/3/7. **No violations.**

Recommendation: Use the same pattern in product code for IO-bound or lock-sharing tasks; re-check this table after every affinity change.

Suspend / resume (test 9) Statistics → Task Lifecycle

Task	Susp/Res	Runs	Alive span
SR0[271] ... SR3[274]	4/4 each	32-50	~13-15 ms

Recommendation: Equal Susp/Res confirms STI lifecycle hooks. On the timeline, confirm suspended subjects do not run between suspend and resume (including after a semaphore give while still suspended).

Tags / intervals Tag Analysis → tag0_event: 2357 samples (min 8 144, avg ≈ 34 631, max 45 392, p95 41 936).

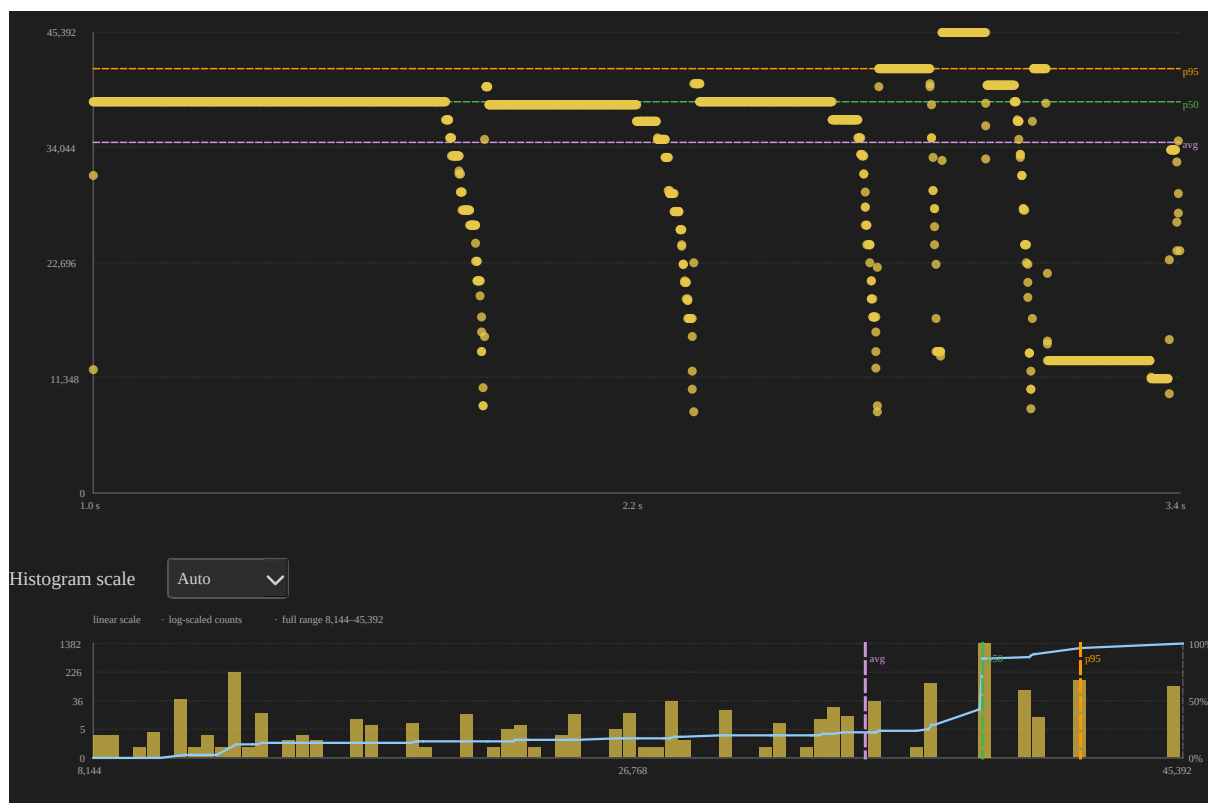


Figure 7: `tag0_event` distribution

Recommendation: Bind tag channels to real budgets (heap high-water, queue depth). Rising max/p95 across builds → Trace Compare on Tag Analysis / Interval Analysis.

3.7 Performance verdict and improvement plan

Priority	Finding	Verdict on this sample	Product-oriented action
P0	CS migration thrash (~1.6k/s, dwell ~0.5 ms)	Expected stress artefact	Affinity-pin hot tasks; reduce equal-priority fan-out; re-measure Migr rate / Ping
P0	Queue/mutes stress + shared objects across cores bounces (queue 858)		Co-locate producers/consumers; shorten holds; consider core-local queues
P1	Priority L/M/H on PS[228]; long High block ~53 ms	Demo inversion geometry	Keep inheritance; audit critical sections; add deadline thresholds
P1	WCET CS Max ~3.6 ms vs 1 ms tick	Stress length	Budget slices; break work; verify under tickful config
P2	TICKLESS CV 35.9 %, missed ≈ 8	Idle between tests	Scope busy windows; only then chase missed ticks
P2	Load Balance 95 %, σ 6 %	Healthy	No balance change
OK	Affinity + SR Susp/Res 4/4	Hooks correct	Keep as regression checks in CI (report HTML)

Suggested next capture (after code changes)

Before/after the same suite

```
python builds/btf_viewer.py compare before.btf.gz after.btf.gz \
  --output compare.html --format html \
  --name-a "Before" --name-b "After"
```

Success criteria worth watching in Δ : lower CS **Migr rate** / **Ping**, lower queue **Bounces**, stable or improved High **Max block**, Load Balance Score still $\geq 85\%$ with $\sigma \leq 30\%$.

4. Deep-Dive Playbooks

Use these when Analysis or the ladder points at a specific metric. Examples below assume example-8cores unless noted.

4.1 Core utilisation and load balance

1. **Statistics → Core Utilisation** — bars are active (non-IDLE, non-TICK) CPU%.
2. Badge **amber** when population $\sigma > 30\%$.
3. **Analysis** warns when Score $< 70\%$ **or** $\sigma > 30\%$; “reasonably balanced” when Score $\geq 85\%$ and $\sigma \leq 30\%$.
4. **Top Tasks by CPU** — who consumes the budget.

Symptom	Next
One core $> 90\%$, others idle	Core Affinity
All high, one low	Mutex/Semaphore, Preemption Chain
Score mid ($\sim 60\%$) but $\sigma \leq 30\%$	Still uneven — Core Migrations

4.2 WCET (execution time per slice)

1. **Execution Time Per Slice** → sort **Max**.
2. Click **Max** — zoom + annotate.
3. Check neighbours on the core (tooltip) and **Preemption Chain** for that victim.

4.3 Blocking / response time

1. **Blocking Time** → sort **Max** / **p95**.
2. Long gap → **Preemption Chain** (who ran instead) and **Mutex/Semaphore** (who held the lock).
3. Scope away from orchestrator sleeps (Runner).

4.4 Priority inversion

1. **Priority Inheritance** (needs `priority_inherit` / `priority_disinherit` STI).
2. Patterns: **Mutex inherit** (kernel boost), **L/M/H, Boost only**.
3. Timeline: red bottom stripe on the holder for the full boost window.
4. Distribution chart → click a point to jump.

4.5 Core migrations

1. **Core Migrations** — **Rate, Dwell, Ping-pong, Lock-bounce** %.
2. **Core View** + filter + lock-highlight the hot task — watch it hop core rows (snapshot ... `--view-mode core --task ...`).
3. **Core-Pair Migration Summary** + toolbar **Heatmap** (32 bins) / **Chord** — for a *specific core*, scan pairs / hover the chord arc where that core is an endpoint.
4. Red flags: high rate + short dwell; any lock-bounce on latency paths.

4.6 Tick health

1. **Trace Health (TICK)** — mode badge, CV, large gaps, missed estimate.
2. **Tick Distribution...** for scatter/histogram.
3. Always re-evaluate inside a **cursor-scoped busy** window on tickless systems.

4.7 Mutex / semaphore correctness

1. **Mutex / Semaphore** — Issues, Bounces, Status.
2. Warning rows → timeline STI take/give; check delete-while-held.
3. High **Core bounce** → pin sharers or redesign ownership.

4.8 Task suspend / resume

1. **Task Lifecycle** — Susp/Res must match per subject (SR* → 4/4 on this sample).
2. Timeline: no run between suspend and resume.
3. Desktop: navigator minimap sits above the CPU Load overlay while zoomed.

4.9 Deadlines and CPU budgets

Settings (Ctrl+,) → **Analysis thresholds**:

Threshold	Example
CPU budget %	25
Task deadlines	CS[28]=2000000 (ns) — slices > 2 ms

Then **Statistics** → **Deadlines / CPU budget**. Use cursor scope per test phase.

5. Scope, Compare, and Custom Signals

5.1 Cursor-scoped analysis

Action	Effect
Left-click	Place / remove cursor
Shift+click	Snap to segment edge

Action	Effect
C	Cursor at viewport centre
Ctrl+R	Zoom to C1...Cn
Limit to cursor range	All Statistics + Analysis recompute for C1-Cn

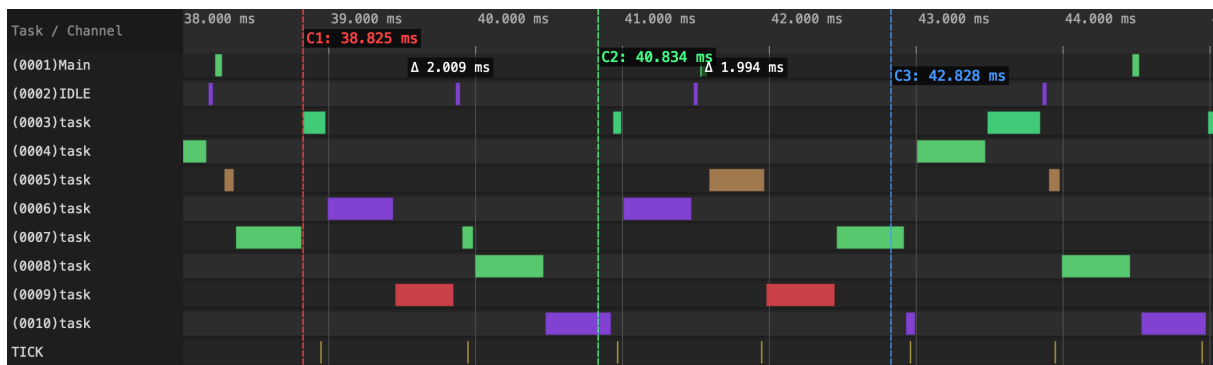


Figure 8: Cursors C1-C3 with Δ badges

Metric	Cursor rule
CPU %	Overlap $\div$ range width
Execution slices	Fully inside range
Blocking	Both slices inside
Migrations	Overlapping events

5.2 Compare two builds

1. Open two tabs $\rightarrow$ Statistics footer **Trace Compare....**
2. Tabs: Summary, Top Tasks, Migrations, Blocking, Preemption, Sync.
3. $\Delta = B - A$. Export CSV/HTML.

```
python builds/btf_viewer.py compare before.btf.gz after.btf.gz \
  --output compare.html --format html \
  --name-a "Before" --name-b "After"
```

5.3 Tags and intervals

```
trace_tag_emit(0, (int)value);           /* tag0_event ... tag7_event */
trace_interval_start(1); /* ... */ trace_interval_stop(1);
```

- Waveform rows + **Tag Analysis / Interval Analysis** charts.
- Notes with tid:{task_id} pair intervals per task.

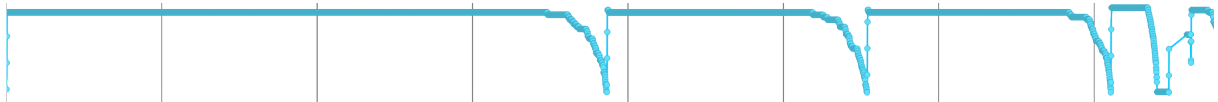


Figure 9: tag0_event waveform

6. Export Results

Analysis Findings

Toolbar **Analysis** (or HTML report card). Typical triggers:

Finding	Trigger
Load imbalance	Score < 70 % or σ > 30 %
Balanced (info)	Score $\geq$ 85 % and $\sigma \leq$ 30 %
WCET candidates	Highest CPU% tasks
Blocking	Many / long off-CPU gaps
Priority inversion	L/M/H pattern
Thrashing	High Migr rate / Ping / short dwell
Tick	Not GOOD, or missed > 0
Sync bounces	Core bounce > 0

GUI and CLI

Output	How
Findings text	Analysis → Save as text...
Statistics CSV/HTML	Statistics → Export
Perfetto JSON	File → Export Perfetto... / perfetto CLI
Timeline PNG/SVG	Shot / Save as SVG
Headless report	<code>report trace.btf --output out.html --format html</code>

```
python builds/btf_viewer.py info ../tracedata/example-8cores.btf.gz
python builds/btf_viewer.py report ../tracedata/example-8cores.btf.gz \
    --output report.html --format html
# --lo/--hi are raw #timeScale units (us for this sample), not nanoseconds
python builds/btf_viewer.py report ../tracedata/example-8cores.btf.gz \
    --output pi.html --format html --lo 3042000 --hi 3359000
python builds/btf_viewer.py migrations ../tracedata/example-8cores.btf.gz
↪ -o mig.csv
# Core View: highlight a task hopping cores
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
    -o migrate.svg --view timeline --view-mode core --task "CS[22]" \
    --lo 1805000 --hi 1865000
python builds/btf_viewer.py perfetto ../tracedata/example-8cores.btf.gz -o
↪ trace.json
```

Quick-Reference: Metric to Root Cause

Observed symptom	Start here	Then check
Unknown — triage first	Toolbar Analysis	Named Statistics sections in each finding
Tick jitter / tickless	Trace Health (TICK)	Scope busy window; Execution Max slices

Observed symptom	Start here	Then check
SMP uneven load	Core Utilisation (Score / σ)	Migrations, Affinity
Task too slow on CPU	Execution Time (Max / p95)	Preemption Chain, Mutex
Task waits too long	Blocking Time	Preemption Chain, Mutex, Inter-Arrival
Priority inversion	Priority Inheritance	Mutex pairing, Blocking
Core thrashing	Core Migrations (Rate, Ping)	Core View highlight (- -view-mode core); Core-Pair / Heatmap / Chord; Bounce %; Affinity
Lock / queue issues	Mutex/Semaphore / Queue	Blocking, Migrations
Suspend/resume	Task Lifecycle (Susp/Res)	Timeline STI; demo test 9
Affinity wrong	Core Affinity	Lock-bounce table
Custom metric / region	Tag / Interval Analysis	Owning task Execution Time
Before/after change	Trace Compare	Same cursor phases on both tabs
